REFINED WEEKLY GOALS TASK BOARD

HAMSINI VANAM – 2520030556
SECTION – 2
CLUSTER – 1

UNDER THE GUIDANCE OF SUPERVISOR NAME:

ASSISTANT PROFESSOR:

DEPARTMENT OF CSE

OUTLINE

- Abstract
- Existing System
- Proposed System
- System Architecture
- Project Modules
- Objectives
- CO Attainments – FWD
- CO Attainments – DSA
- Screenshots
- Conclusion

ABSTRACT

The Weekly Planner is a minimalist productivity web application designed to help users organize tasks efficiently across a seven-day layout. The interface focuses on clarity and visual balance, presenting each day as an individual column where tasks can be added, prioritized, completed, or removed. A subtle visual system using typography, color accents, and motion enhances usability while maintaining a clean, distraction-free experience. The planner also includes weekly navigation and a progress summary, enabling users to monitor workload distribution and overall completion rates.

From a technical perspective, the application is built using HTML, CSS, and JavaScript, emphasizing responsive design and smooth interaction. Tasks are stored using localStorage, ensuring data persistence without requiring a backend. Dynamic rendering allows real-time updates when tasks are modified, while features such as priority tagging, completion tracking, and progress visualization contribute to an intuitive workflow. Overall, the project demonstrates practical front-end development skills, combining aesthetic design with functional task management principles.

EXISTING SYSTEM

1. Traditional planners (e.g., Google Tasks) use flat arrays/DB rows with no structural ADT complexity.
2. Tasks stored in insertion order; no built-in priority management or pointer-based traversal.
3. Week navigation requires costly DB queries rather than O(1) HashMap or O(n) list traversal.
4. Undo is absent or one-step only; no explicit Stack data structure backing the history.
5. No algorithm transparency: sorting strategy, Big-O costs, and ADT internals hidden from users.
6. Existing tools cannot demonstrate linked-list reversal, cycle detection, or sort algorithm comparison.
7. Limitation: no side-by-side Insertion Sort $O(n^2)$ vs Merge Sort $O(n \log n)$ demo capability.

PROPOSED SYSTEM

- Replace flat arrays with a custom DoublyLinkedList per day: O(1) head/tail insert, O(n) search and O(1) pointer-splice delete.
- Use a HashMap<String, TaskList> to map date strings (yyyy-MM-dd) to task lists — O(1) average-case lookup and insertion.
- Implement an explicit Stack (ArrayDeque) for multi-level undo: push on every ADD / DELETE / TOGGLE, pop to reverse the last action.
- Integrate a PriorityQueue (min-heap) to display tasks in descending priority order — O(n log n) build, O(log n) per poll.
- Provide two sorting algorithm implementations (Insertion Sort $O(n^2)$ vs Merge Sort O(n log n)) with a live comparison demo.
- Add Binary Search (O(log n)) on a sorted-by-ID array, contrasted with linear search (O(n)), to illustrate search efficiency.
- Build a web front-end (HTML/CSS/JS) mirroring the same planner logic with priority colour-coding, progress bar, and week navigation.

SYSTEM ARCHITECTURE

USER INTERFACE (FRONT END)

↓

BROWSER LOGIC (JAVASCRIPT)

↓

LOCAL STORAGE (DATA PERSISTENCE)

SYSTEM ARCHITECTURE DIAGRAM

FRONT END
(HTML, CSS, JAVASCRIPT)

BROWSER LOGIC
(JAVASCRIPT)

LOCAL STORAGE
(DATA PERSISTENCE)

PROJECT MODULES

- Module 1 — Task Management Core: Add, delete, and toggle-complete tasks; Task class with id, text, priority, done flag.
- Module 2 — DoublyLinkedList ADT: addLast O(1), findById O(n), deleteById O(1) splice, reverse O(n), hasCycle O(n).
- Module 3 — HashMap Date Index: taskMap backed by Java HashMap; O(1) avg insert/lookup; computeIfAbsent for lazy init.
- Module 4 — Undo Stack: ArrayDeque used as LIFO stack; UndoAction records ActionType (ADD/DELETE/TOGGLE), date, and task snapshot.
- Module 5 — Priority Display: PriorityQueue with Comparator on priority.level desc; renders tasks highest-first each view.
- Module 6 — Sort & Search Demo: insertionSort $O(n^2)$, mergeSort O(n log n), binarySearchById O(log n) with live console output.
- Module 7 — Web Front-End: 7-column grid planner, per-day task add/delete/complete, priority buttons, weekly progress bar.

OBJECTIVES

- CO1 — Implement and compare sorting algorithms: Insertion Sort $O(n^2)$ vs Merge Sort $O(n \log n)$; Binary Search $O(\log n)$ vs Linear $O(n)$.
- CO2 — Build ADTs using arrays and linked lists: custom DoublyLinkedList with insert, delete, search, reverse, and cycle detection.
- CO3 — Apply Stack (undo history) and PriorityQueue (priority-ordered display) in a real-world planning context.
- CO4 — Use HashMap as a hash-table data structure mapping date keys to task lists with O(1) average-case operations.
- CO5 — Develop a complete, practical weekly planner application with statistics, progress bar, week navigation, and undo.
- CO6 — Produce a runnable, well-documented Java program integrating all data structures with clear CO mapping comments.

Bonus — Complement the Java app with a web-based version (HTML/CSS/JS) demonstrating the same planner logic visually.

CO'S ATTAINMENTS - FWD

- CO1 — Identified and analysed time complexity: O(n) search, O(1) insert, O(n log n) sort in JS planner.
- CO2 — Applied array-backed lists and linked structures to manage weekly task data in HTML/JS front-end.
- CO3 — Used stack-like undo pattern and queue-ordered task rendering in the web planner event model.
- CO4 — Used JS object as hash map for date-keyed task storage; O(1) average-case lookup by date string.
- CO5 — Delivered full web planner: add/delete/complete tasks, priority buttons, progress bar, week navigation.

CO'S ATTAINMENTS - DSA

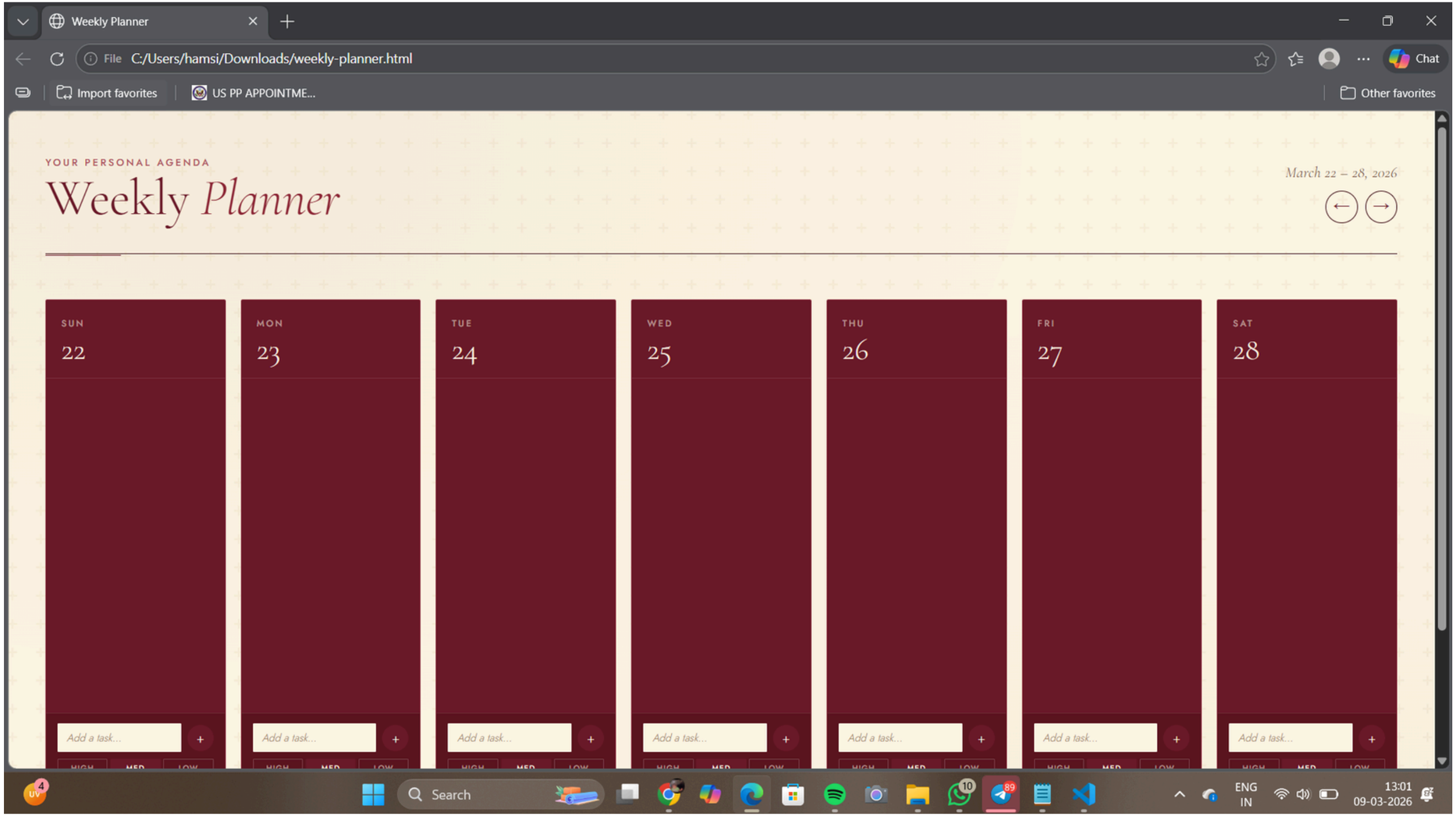
- CO1 — Big-O documented per operation: O(1) add, O(n) search, O(n log n) PQ, O(n^2)/O(n log n) sort.
- CO2 - Custom DoublyLinkedList<Task>: addLast O(1), findById O(n), deleteById O(1) splice, reverse O(n).
- CO3 — ArrayDeque as LIFO undo stack O(1) push/pop; PriorityQueue heap for priority-ordered display.
- CO4 — Java HashMap<String, TaskList>: computeIfAbsent lazy init; O(1) avg lookup; hash-table design.
- CO5 — Java console planner: add/delete/toggle, multi-level undo, sort demo, binary search, week navigation.

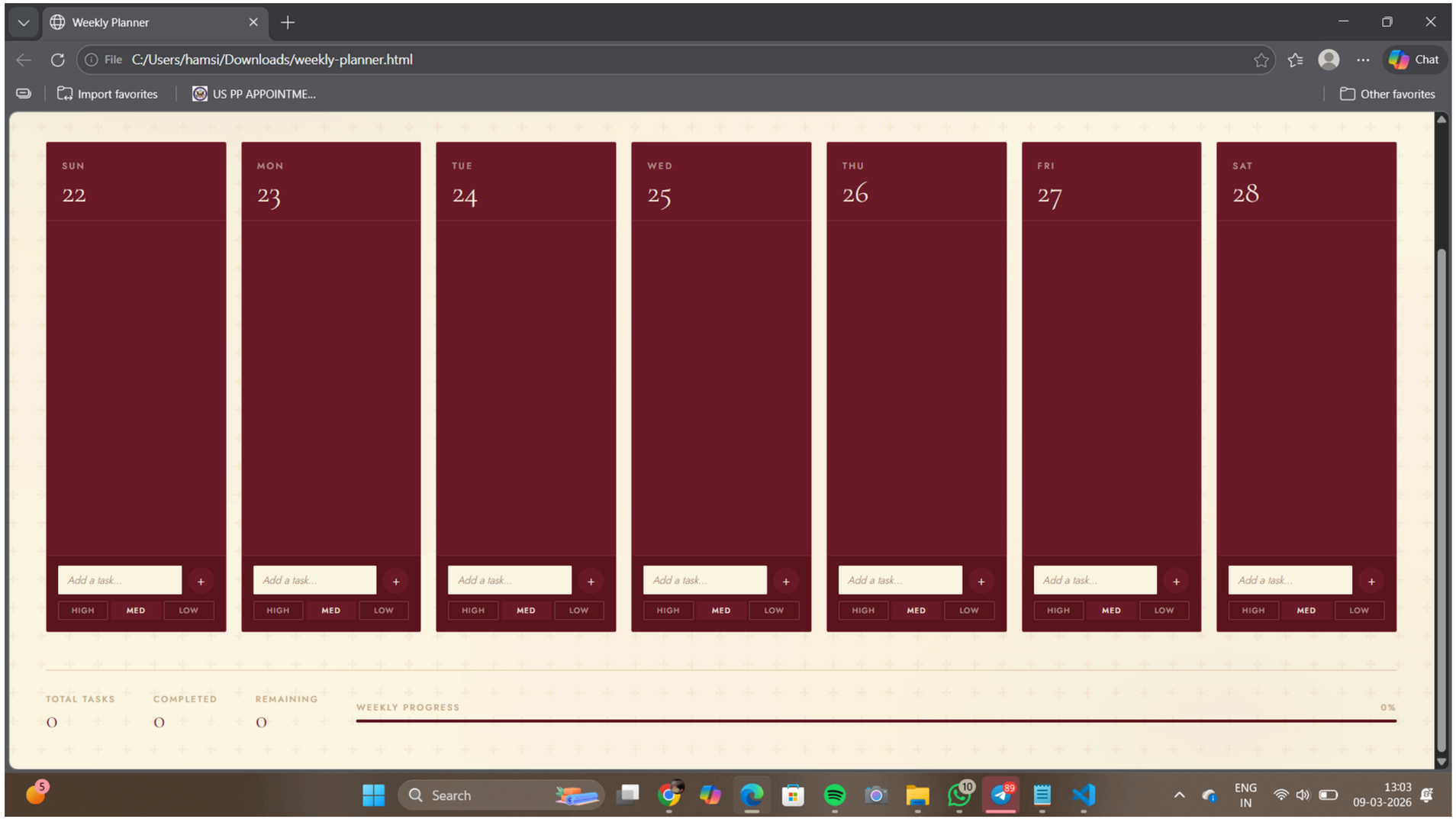
PROJECT DESIGN

Design Focus:

- User-friendly interface
- Smooth task interaction
- Visual priority indicators

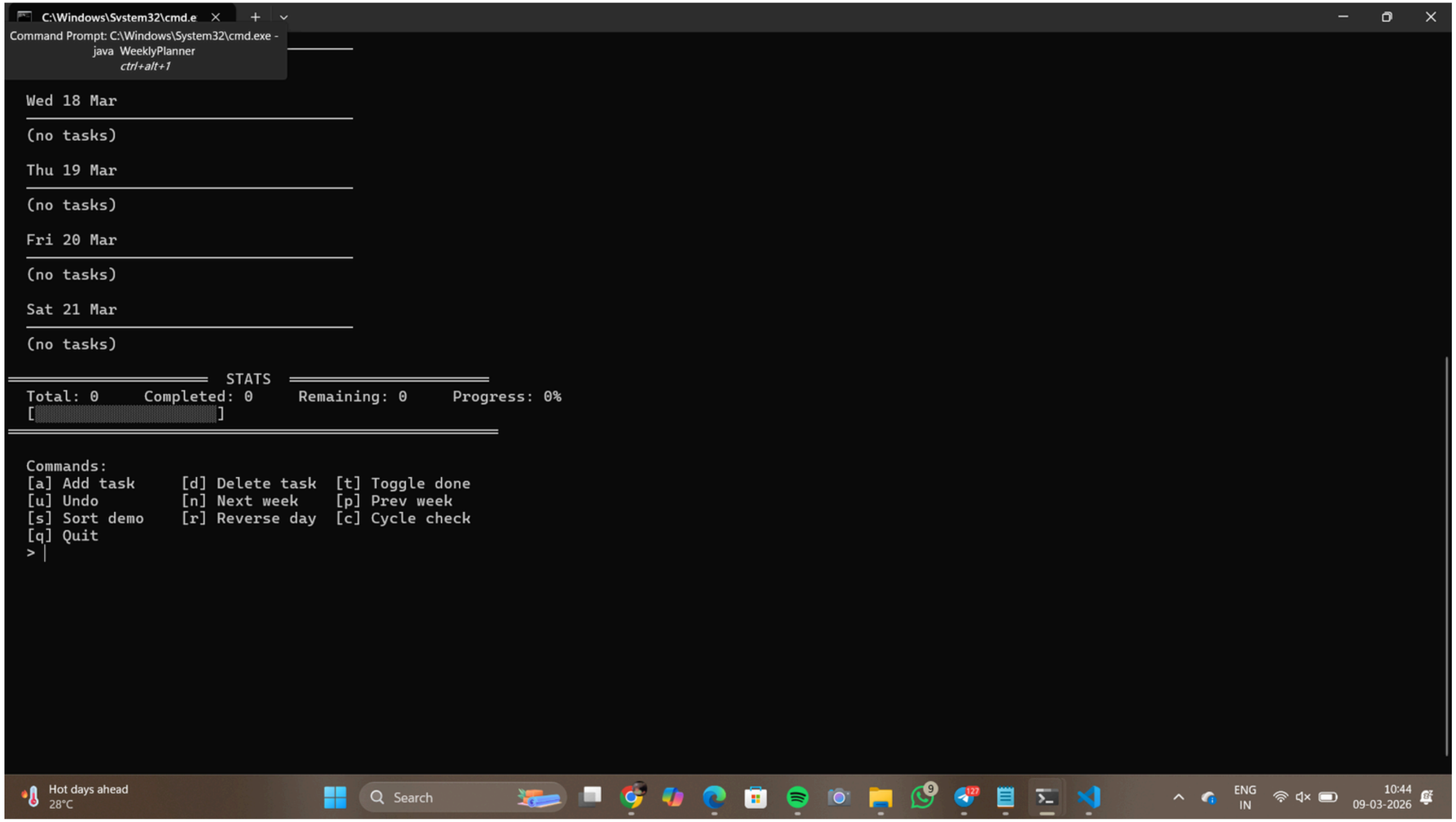
SCREENSHOTS OF FWD




Weekly Planner
File C:/Users/hamsi/Downloads/weekly-planner.html
Chat
Import favorites
US PP APPOINTME...
Other favorites
SUN 22
MON 23
TUE 24
WED 25
THU 26
FRI 27
SAT 28
Add a task...
HIGH MED LOW
TOTAL TASKS 0
COMPLETED 0
REMAINING 0
WEEKLY PROGRESS 0%
Search
ENG IN
13:03 09-03-2026

SCREENSHOTS OF DSA

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\DSA Project>javac WeeklyPlanner.java

C:\DSA Project>
C:\DSA Project>java WeeklyPlanner
  ? Added: [ ] #1   [HIGH]   Review lecture notes
  ? Added: [ ] #2   [HIGH]   Complete assignment 2
  ? Added: [ ] #3   [MED]    Read chapter 5
  ? Added: [ ] #4   [LOW]    Exercise
  ? Added: [ ] #5   [HIGH]   Submit lab report
  ? Added: [ ] #6   [MED]    Team meeting prep

  == Welcome to Weekly Planner (Java) ==
  Data Structures & Algorithms Edition


 ╔═══════════════════════════════════════╗
 ╚  WEEKLY PLANNER  ?  2026-03-15 to 2026-03-21   ╝


  Sun 15 Mar
  ──────────────────────────────
  (no tasks)

  Mon 16 Mar
  ──────────────────────────────
  (no tasks)

  Tue 17 Mar
  ──────────────────────────────
  (no tasks)

  Wed 18 Mar
  ──────────────────────────────
  (no tasks)

  Thu 19 Mar
  ──────────────────────────────
  (no tasks)
```


C:\Windows\System32\cmd.e
Command Prompt: C:\Windows\System32\cmd.exe - java WeeklyPlanner
ctrl+alt+1
Wed 18 Mar
(no tasks)
Thu 19 Mar
(no tasks)
Fri 20 Mar
(no tasks)
Sat 21 Mar
(no tasks)
STATS
Total: 0 Completed: 0 Remaining: 0 Progress: 0%
[ ]
Commands:
[a] Add task [d] Delete task [t] Toggle done
[u] Undo [n] Next week [p] Prev week
[s] Sort demo [r] Reverse day [c] Cycle check
[q] Quit
>
Hot days ahead
28°C
Search
ENG
IN
10:44
09-03-2026

CONCLUSION

- Integrated six core ADTs: DoublyLinkedList, HashMap, Stack (Deque), PriorityQueue, Array, and heap.
- All six Course Outcomes CO1–CO6 explicitly demonstrated with Big-O annotations and runnable comparisons.
- Java console app provides hands-on pointer manipulation, hash-table design, and stack-based multi-level undo.
- HTML/CSS/JS web companion validates that the same algorithms translate directly to front-end development.
- Insertion Sort vs Merge Sort live demo gives learners a tangible $O(n^2)$ vs $O(n \log n)$ performance contrast.
- Floyd's cycle detection and linked-list reversal demonstrate $O(n)$ time / $O(1)$ space elegant algorithm design.
- Future: recurring tasks, drag-and-drop reordering (merge-sort visualiser), cloud sync via REST API.

GEO-TAG PHOTO

THANK YOU